

ALG-CLTAV-01 wrapper

CL-TAV specification (CR-2026-013 / DD-033)

This wrapper exists only to compile `ALG-CLTAV-01.tex`. It is not a second algorithm. The procedure is the typeset form of the method-report contract. It is not an executable engine and does not select solver technology. Step labels S0–S10 are the stable bilingual mapping; typeset line numbers are not cited.

Chinese step groups (same process, not a second program)

- S0: Γ 为唯一状态源; q / `retry` / $R_{\max}$ 是 Γ .qStatus / `retryCount` / `retryCap` 的别名; 初始化 η 、 H_0 与 `retry`。
- S1: 停止 / 错误优先门; 已决定的 A2–A5 或 P1–P5 立即返回。
- S2: 选择前预测; `PredictionGapError` 只关掉 TEST 评分; UNKNOWN 且 Recover 仍合格时不得挡合法恢复。
- S3: 互斥选择 TEST minimax / `Prep` / `Recover` / `Admit`; 空 TEST 集不得 `arg min`; 空 $\mathcal{C}$ 不阻止合格 Recover。选择后、任何写接口之前冻结 $qUsedAtSelect$ 、历史版本、预测类与动作身份。
- S4: 一次计费; 重试上限不是预算耗尽。决策索引 k 、计费次数与 `retry` 彼此不同。
- S5–S6: 执行与解释返回效果类 CONFIRMED-NOT-SENT 或 UNKNOWN-EFFECT, 不把二者合成同一种未知。
- S7: `prepErr` 与 UNKNOWN-EFFECT 将 η 标为保守未知并把 q 写成 UNKNOWN; CONFIRMED-NOT-SENT 保持 q 与相容历史并仍计重试; 未确认 Recover 保持 UNKNOWN。达 $R_{\max}$ 返回 Stop-Error。
- S8: 仅主过程把已确认 Recover 写成 KNOWN (声明目标与依据)。IF-PREP-RECOVER 只返回值, 不写 Γ 。
- S9: 历史更新读选择时快照 $qUsedAtSelect$ 与单独的 *postSummary*, 不读 S8 写后的 q 。
- S10: 互斥 P1–P5; 未停止则下一轮从 S1 继续。未确认恢复回边在 S7, 不代替这条正常循环。

Stop-645 在本增量之后表示仍未在能力/实现层关闭的具名 645 依赖义务, 不是“645 文件缺失”。

Algorithm 1: CL-TAV top-level session (ALG-CLTAV-01)

Require: declared $H_0 = \{h_{\text{normal}}\} \cup H_{\text{single}}$; SessionContext Γ (unique state); HistoryHandle η ; resource mode $\text{BUDGET}(c_{\min}, B) \text{ xor } \text{ROUNDS}(K_{\max})$; action library; measurement uncertainty

Ensure: stop class, final H , η , session trace; singleton h_{normal} is not protocol PASS

// S0 $H \leftarrow H_0$; $q := \Gamma.\text{currentSummary}$, $\text{retry} := \Gamma.\text{retryCount}$, $R_{\max} := \Gamma.\text{retryCap}$; $\Gamma.\text{retryCount} \leftarrow 0$; $\Gamma.\text{qStatus}$ and q UNKNOWN unless declared; η empty. Decision index k , billed charges, and retry are distinct

```

1 while true do
  // S1 stop gate before prediction; an already decided stop returns here
2  if IF-RESOURCE-STOP( $\Gamma, H, \eta$ ) already decided Admit A2-A5 or P1-P5 then return that stop,  $H, \eta$ , trace;
  // S2 predict before select; PredictionGapError is TEST-scoped
3  ( $\text{pred}, C$ )  $\leftarrow$  IF-PRED-OBS( $\Gamma, \eta, H$ , uncertainty, library);
4  if  $\text{pred}$  is PredictionGapError then
5    TEST scoring unavailable;  $C \leftarrow \emptyset$ ;
6    if Recover and Prep are not eligible from  $\Gamma$  then return named prediction-gap; do not score empty  $C$  as remaining
       |  $H$ ;
7   $A \leftarrow$  currently enabled actions;
  // S3 exclusive kind; empty  $C$  does not block eligible Recover or Prep
8  ( $\text{kind}, t^*, S, \text{admit}$ )  $\leftarrow$  IF-SELECT-ADMIT( $A, \Gamma.\text{currentSummary}, C, \Gamma, \eta, H$ );
9  if  $\text{kind} = \text{ADMIT}$  then
10   | return  $\text{admit} \in \{A2, A3, A4, A5\}, H, \eta$ , trace;
11  else if  $\text{kind} = \text{TEST}$  then
12   | if  $S = \emptyset$  then return named spec error; do not arg min empty  $S$ ;
13   |  $t^*$  is the final one-step minimax action selected by IF-SELECT-ADMIT on currently valid nonempty classes  $C$  inside
       | TEST-only  $S$ ;
14  else if  $\text{kind} \in \{\text{PREP}, \text{RECOVER}\}$  then
15   | keep the Prep or Recover  $t^*$  named by IF-SELECT-ADMIT; do not enter TEST minimax;
16  else
17   | return named spec error;
  // S3-SNAP freezes the final non-ADMIT action only after all selection rules
18   $q_{\text{UsedAtSelect}} \leftarrow \Gamma.\text{currentSummary}$ ;  $\text{histVer} \leftarrow \eta.\text{version}$ ;  $\text{classesUsedAtSelect} \leftarrow C$ ;  $\text{actionId} \leftarrow t^*$ ;
  // S4 charge once; retry cap  $\neq$  budget exhaustion
19  charge  $t^*$  once under the selected mode; refuse unadmitted issue;
  // S5 execute; the effect class is returned
20  ( $\text{record}, \text{effect}$ )  $\leftarrow$  IF-EXECUTE-RECORD( $\Gamma, t^*$ );
  // S6 interpret unless execute already returned an effect class
21   $I_z \leftarrow \text{nil}$ ;  $\text{confirmed} \leftarrow \text{false}$ ;  $\text{prepErr} \leftarrow \text{nil}$ ;
22   $\text{postSummary} \leftarrow \text{nil}$ ;  $\text{declaredTarget} \leftarrow \text{nil}$ ;  $\text{evidence} \leftarrow \text{nil}$ ;
23  if  $\text{effect} \in \{\text{CONFIRMED-NOT-SENT}, \text{UNKNOWN-EFFECT}\}$  then
24   | skip interpret;
25  else
26   | ( $I_z, \text{effect}, \text{postSummary}$ )  $\leftarrow$  IF-OBS-INTERPRET( $\Gamma, \text{record}$ );
27  if  $\text{kind} \in \{\text{PREP}, \text{RECOVER}\}$  and  $\text{effect} \notin \{\text{CONFIRMED-NOT-SENT}, \text{UNKNOWN-EFFECT}\}$  then
28   | ( $\text{confirmed}, \text{prepErr}, \text{declaredTarget}, \text{evidence}, \text{postSummary}$ )  $\leftarrow$  IF-PREP-RECOVER( $\Gamma, \eta, t^*, \text{record}$ );
       | // IF-PREP-RECOVER returns values; it does not write  $\Gamma$  or  $\eta$ 
29  if  $\text{effect} \in \{\text{CONFIRMED-NOT-SENT}, \text{UNKNOWN-EFFECT}\}$  or  $\text{prepErr}$  or ( $t^*$  is Recover and  $\text{confirmed}$  is false) then
       | // S7 classify the effect; do not fold CONFIRMED-NOT-SENT into UNKNOWN
30   | do not exclude  $H$ ;
31   | if  $\text{prepErr}$  or  $\text{effect} = \text{UNKNOWN-EFFECT}$  then
32   | | mark  $\eta$  conservative-unknown;  $\Gamma.\text{qStatus} \leftarrow \text{UNKNOWN}$ ; clear  $\Gamma.\text{currentSummary}$ ;
33   | else if  $\text{effect} = \text{CONFIRMED-NOT-SENT}$  then
34   | | leave  $\Gamma.\text{qStatus}$  and compatible history unchanged;
35   | else
       | // unconfirmed Recover
36   | |  $\Gamma.\text{qStatus}$  stays UNKNOWN; clear  $\Gamma.\text{currentSummary}$ ; mark  $\eta$  conservative-unknown;
37   |  $\Gamma.\text{retryCount} \leftarrow \Gamma.\text{retryCount} + 1$ ;
38   | if  $\Gamma.\text{retryCount} \geq \Gamma.\text{retryCap}$  then return Stop-Error; this is not A5/Stop-Budget;
39   | continue after the one charge;
40  else if  $t^*$  is Recover then
       | // S8 sole writer of KNOWN( $\text{declaredTarget}, \text{evidence}$ )
41   |  $\Gamma.\text{qStatus} \leftarrow \text{KNOWN}(\text{declaredTarget}, \text{evidence})$ ;
42   |  $\text{postSummary} \leftarrow (\text{declaredTarget}, \text{evidence})$ ;
43  if  $I_z$  is a valid observation class or  $\text{postSummary}$  is confirmed then
       | // S9 history reads the select snapshot; this is the sole commit of a confirmed successor summary
44   | ( $\eta, H$ )  $\leftarrow$  IF-HIST-UPDATE( $\eta, H, t^*, q_{\text{UsedAtSelect}}, \text{classesUsedAtSelect}, I_z, \text{postSummary}$ );
45   | if  $\Gamma.\text{qStatus}$  is KNOWN and  $\text{postSummary}$  is confirmed then
46   | |  $\Gamma.\text{currentSummary} \leftarrow \text{postSummary}$ ;
       |
       | // S10 exclusive P-stops; if none, the next iteration starts at S1
47  let  $\sigma \leftarrow$  first of Stop-Empty, Stop-Singleton, Stop-645, Stop-Equivalent, Stop-Budget via IF-RESOURCE-STOP( $\Gamma, H, \eta$ ) and
       | IF-EQUIV( $\Gamma, H, \eta$ );
48  if  $\sigma$  is set then return  $\sigma, H, \eta$ , and the trace;

```
